МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

Лабораторная работа № 5

по дисциплине «Математические основы баз данных»

по теме: «Транзакции, целостность данных и серверная логика.»

Выполнил студент группы:
БВТ2505
Пондяков Игорь Дмитриевич
Проверил:
Быватов Семён Юрьевич

Москва 2026

Цели и задачи

Цель: освоить механизмы обеспечения целостности данных и реализации серверной бизнес-логики в СУБД PostgreSQL. Научиться управлять транзакциями для защиты критичных операций от сбоев, инкапсулировать сложные алгоритмы в пользовательские функции на языке PL/pgSQL, а также автоматизировать рутинные операции и аудит с помощью триггеров.

Задачи:

1. Управление транзакциями: Разработать скрипт транзакции, моделирующий критичный бизнес-процесс провайдера (например, смена тарифного плана с проверкой статуса и перерасчётом баланса). Использовать команды BEGIN, COMMIT, ROLLBACK и SAVEPOINT для обеспечения атомарности и обработки ошибок на промежуточных этапах.
2. Серверные функции (PL/pgSQL): Реализовать пользовательскую функцию, инкапсулирующую бизнес-логику (например, автоматическое создание заявки в техподдержку с назначением свободного инженера). Функция должна использовать локальные переменные, условные операторы (IF/ELSE) и механизм обработки исключений (EXCEPTION) для прерывания выполнения при нарушении бизнес-правил.
3. Триггер аудита: Создать триггер и таблицу журнала (audit_log) для автоматического протоколирования изменений исторически важных данных (например, изменения статуса договора или стоимости тарифа). Триггер должен сохранять старые (OLD) и новые (NEW) значения, а также метку времени и пользователя.
4. Триггер для денормализованного поля: Реализовать триггер, который автоматически поддерживает актуальность денормализованного поля. Например, при добавлении или удалении услуги в таблице contract_services триггер должен автоматически пересчитывать и обновлять итоговую месячную стоимость договора в таблице contracts.
5. Архитектурное обоснование: Сформулировать и задокументировать критерии разделения логики: какие задачи уместно решать на уровне базы данных (триггеры, функции), а какие — выносить на уровень приложения.

Выполнение

1. Транзакции: COMMIT, ROLLBACK и SAVEPOINT

Продемонстрировать три ключевых механизма управления транзакциями в PostgreSQL на примере бизнес-процесса «Смена тарифного плана»:

- Успешная транзакция с COMMIT — фиксация всех изменений.
- Транзакция с ошибкой и полным ROLLBACK — откат всех изменений при нарушении бизнес-правила.
- Частичный откат через SAVEPOINT — сохранение части изменений при ошибке на промежуточном этапе.

Успешная транзакция (COMMIT)

Бизнес-правило: сменить тариф у активного договора №1 с текущего на «Геймер» (tariff_id = 3). Операция должна быть атомарной: либо тариф меняется, либо нет.

```
SELECT contract_id, subscriber_id, tariff_id, status
FROM contracts WHERE contract_id = 1;

BEGIN;
   SELECT status FROM contracts
   WHERE contract_id = 1 FOR UPDATE;

   UPDATE contracts
   SET tariff_id = 3,
      start_date = CURRENT_DATE
   WHERE contract_id = 1;

   SELECT 'Тариф успешно изменён на Геймер' AS result;
COMMIT;

SELECT contract_id, tariff_id, start_date
FROM contracts WHERE contract_id = 1;
```

	contract_id [PK] integer	tariff_id integer	start_date date
1	1	3	2026-10-16

Транзакция с ошибкой и полным ROLLBACK

Бизнес-правило: пытаемся сменить тариф у закрытого договора. Операция должна быть полностью отменена.

```
UPDATE contracts SET status = 'closed' WHERE contract_id = 2;

SELECT contract_id, tariff_id, status FROM contracts WHERE contract_id = 2;

BEGIN;

DO $$
DECLARE
   v_status VARCHAR(20);
BEGIN
   SELECT status INTO v_status
   FROM contracts WHERE contract_id = 2 FOR UPDATE;

   IF v_status != 'active' THEN
      RAISE EXCEPTION 'Нельзя сменить тариф: договор 2 имеет статус %',
v_status;
   END IF;
END $$;

UPDATE contracts
SET tariff_id = 4
WHERE contract_id = 2;

COMMIT;
```

	contract_id [PK] integer	tariff_id integer	status character varying (20)
1	2	2	active

(После ручного ввода ROLLBACK; и SELECT contract_id, tariff_id, status FROM contracts WHERE contract_id = 2;)

Частичный откат через SAVEPOINT

Продемонстрировать механизм обработки исключений в PL/pgSQL для реализации частичного отката внутри транзакции. В отличие от классического SAVEPOINT, который требует выполнения всего блока одной командой, подход с блоком DO $$ ... EXCEPTION ... END $$ позволяет перехватывать ошибки и продолжать выполнение транзакции даже в режиме автокоммита pgAdmin.

```
BEGIN;

UPDATE contracts
SET tariff_id = 2
WHERE contract_id = 3;

DO $$
BEGIN
   INSERT INTO subscribers (full_name, passport, phone)
   VALUES ('Тестовый Абонент', '1234 000001', '+79999999999');
EXCEPTION
   WHEN unique_violation THEN
      RAISE NOTICE 'Ошибка уникальности: дубликат паспорта.
Пропускаем вставку.';
END $$;

UPDATE contracts
SET tariff_id = 3
WHERE contract_id = 4;

COMMIT;
```

Договор №3 получил tariff_id = 2 (изменение сохранилось)
Договор №4 получил tariff_id = 3 (новое изменение после обработки ошибки)
Вставка дубликата паспорта была отклонена с уведомлением

2. Функция на PL/pgSQL для подключения услуги к договору

Создать серверную функцию fn_connect_service, которая инкапсулирует бизнес-логику подключения дополнительной услуги к активному договору. Функция проверяет статус договора, отсутствие

дубликата подключения, добавляет услугу в таблицу contract_services и возвращает итоговую месячную стоимость договора с учётом всех подключённых услуг.

```
CREATE OR REPLACE FUNCTION fn_connect_service(
    p_contract_id INTEGER,
    p_service_id INTEGER
)
RETURNS NUMERIC(10, 2) AS $$
DECLARE
    v_contract_status VARCHAR(20);
    v_service_price NUMERIC(10, 2);
    v_base_tariff_price NUMERIC(10, 2);
    v_total_price NUMERIC(10, 2);
    v_already_connected BOOLEAN;
BEGIN
    SELECT status INTO v_contract_status
    FROM contracts
    WHERE contract_id = p_contract_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Договор % не найден', p_contract_id;
    END IF;

    IF v_contract_status != 'active' THEN
        RAISE EXCEPTION 'Нельзя подключить услугу: договор % имеет
статус %',
            p_contract_id, v_contract_status;
    END IF;

    SELECT monthly_price INTO v_service_price
    FROM services
    WHERE service_id = p_service_id;

    IF NOT FOUND THEN
        RAISE EXCEPTION 'Услуга % не найдена', p_service_id;
    END IF;

    SELECT EXISTS(
```

```
        SELECT 1 FROM contract_services
        WHERE contract_id = p_contract_id AND service_id = p_service_id
    ) INTO v_already_connected;

    IF v_already_connected THEN
        RAISE EXCEPTION 'Услуга % уже подключена к договору %',
            p_service_id, p_contract_id;
    END IF;

    INSERT INTO contract_services (contract_id, service_id, connected_at)
    VALUES (p_contract_id, p_service_id, CURRENT_TIMESTAMP);

    SELECT t.monthly_price INTO v_base_tariff_price
    FROM contracts c
    JOIN tariffs t ON c.tariff_id = t.tariff_id
    WHERE c.contract_id = p_contract_id;

    SELECT COALESCE(SUM(s.monthly_price), 0) INTO v_total_price
    FROM contract_services cs
    JOIN services s ON cs.service_id = s.service_id
    WHERE cs.contract_id = p_contract_id;

    v_total_price := v_base_tariff_price + v_total_price;

    RETURN v_total_price;

EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Ошибка при подключении услуги: %', SQLERRM;
        RAISE;
END;
$$ LANGUAGE plpgsql;
```

Вызовы функции через успешный SELECT fn_connect_service(1, 1) AS total_monthly_price; и неуспешный дубликат SELECT fn_connect_service(1, 1);

	total_monthly_price numeric
1	1000.00

```
ЗАМЕЧАНИЕ:  Ошибка при подключении услуги: Услуга 1 уже подключена к договору 1

ERROR:  Услуга 1 уже подключена к договору 1
CONTEXT:  функция PL/pgSQL fn_connect_service(integer,integer), строка 39, оператор RAISE

ОШИБКА:  Услуга 1 уже подключена к договору 1
SQL state: P0001
```

	contract_id integer	service_name character varying (100)	connected_at date
1	1	Антивирус	2026-10-17

3. Функция, возвращающая сводный показатель

Реализовать серверную функцию fn_revenue_for_period, возвращающую общую выручку провайдера за указанный период. Функция должна использовать локальные переменные для хранения промежуточных результатов (общая сумма, количество платежей, средний чек), выводить подробную статистику через RAISE NOTICE и возвращать итоговое числовое значение.

```
CREATE OR REPLACE FUNCTION fn_revenue_for_period(
    p_start_date DATE,
    p_end_date DATE
)
RETURNS NUMERIC(12, 2) AS $$
DECLARE
    v_total_revenue NUMERIC(12, 2);
    v_payment_count INTEGER;
    v_avg_payment NUMERIC(10, 2);
BEGIN
    -- Считаем общую выручку за период
    SELECT COALESCE(SUM(amount), 0) INTO v_total_revenue
```

```
    FROM payments
    WHERE payment_date::date BETWEEN p_start_date AND p_end_date;

    SELECT COUNT(*) INTO v_payment_count
    FROM payments
    WHERE payment_date::date BETWEEN p_start_date AND p_end_date;

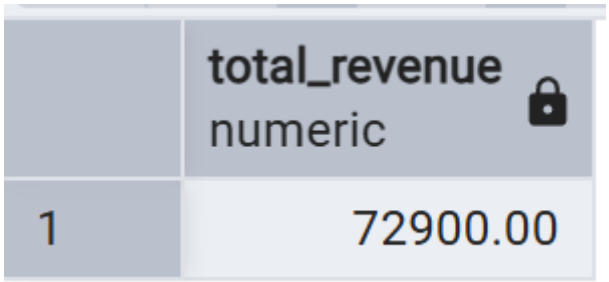
    IF v_payment_count > 0 THEN
        v_avg_payment := v_total_revenue / v_payment_count;
    ELSE
        v_avg_payment := 0;
    END IF;

    RAISE NOTICE 'Период: % — %', p_start_date, p_end_date;
    RAISE NOTICE 'Общая выручка: % руб.', v_total_revenue;
    RAISE NOTICE 'Количество платежей: %', v_payment_count;
    RAISE NOTICE 'Средний платёж: % руб.', v_avg_payment;

    RETURN v_total_revenue;
END;
$$ LANGUAGE plpgsql;
```

Вызов

```
SELECT fn_revenue_for_period('2025-11-01', '2025-11-30') AS total_revenue;
```



	total_revenue numeric
1	72900.00

4. Триггер аудита изменений договоров

Создать таблицу журнала audit_log и триггер, который автоматически записывает все изменения в таблице contracts (изменение статуса, тарифа, даты). Триггер срабатывает после каждого UPDATE и сохраняет старые

(OLD) и новые (NEW) значения, метку времени и имя пользователя, выполнившего изменение.

Создание таблицы журналов

```
CREATE TABLE IF NOT EXISTS audit_log (
   audit_id SERIAL PRIMARY KEY,
   table_name VARCHAR(50) NOT NULL,
   record_id INTEGER NOT NULL,
   action_type VARCHAR(10) NOT NULL,  -- 'INSERT', 'UPDATE',
'DELETE'
   changed_at TIMESTAMP WITH TIME ZONE DEFAULT
CURRENT_TIMESTAMP,
   changed_by VARCHAR(50) DEFAULT current_user,
   old_values JSONB,
   new_values JSONB
);
```

Создание триггерной функции

```
CREATE OR REPLACE FUNCTION fn_audit_contracts()
RETURNS TRIGGER AS $$
BEGIN
   IF TG_OP = 'UPDATE' THEN
      INSERT INTO audit_log (table_name, record_id, action_type, old_values,
new_values)
      VALUES (
         TG_TABLE_NAME,
         OLD.contract_id,
         TG_OP,
         to_jsonb(OLD),
         to_jsonb(NEW)
      );
      RETURN NEW;
   ELSIF TG_OP = 'INSERT' THEN
      INSERT INTO audit_log (table_name, record_id, action_type, new_values)
      VALUES (
         TG_TABLE_NAME,
         NEW.contract_id,
```

```
        TG_OP,
        to_jsonb(NEW)
    );
    RETURN NEW;
  ELSIF TG_OP = 'DELETE' THEN
    INSERT INTO audit_log (table_name, record_id, action_type, old_values)
    VALUES (
        TG_TABLE_NAME,
        OLD.contract_id,
        TG_OP,
        to_jsonb(OLD)
    );
    RETURN OLD;
  END IF;
  RETURN NULL;
END;
$$ LANGUAGE plpgsql;
```

Создание триггера

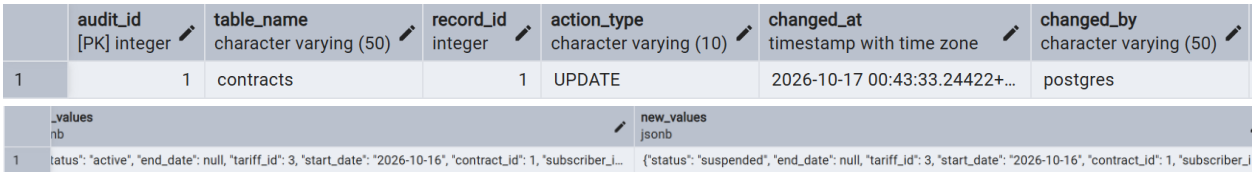
```
CREATE TRIGGER trg_audit_contracts
AFTER INSERT OR UPDATE OR DELETE ON contracts
FOR EACH ROW
EXECUTE FUNCTION fn_audit_contracts();
```

Демонстрация работы

```
SELECT contract_id, status, tariff_id FROM contracts WHERE contract_id = 1;

UPDATE contracts
SET status = 'suspended'
WHERE contract_id = 1;

SELECT audit_id, table_name, record_id, action_type, changed_at, changed_by,
old_values, new_values
FROM audit_log
ORDER BY audit_id DESC
LIMIT 1;
```



	audit_id [PK] integer	table_name character varying (50)	record_id integer	action_type character varying (10)	changed_at timestamp with time zone	changed_by character varying (50)
1	1	contracts	1	UPDATE	2026-10-17 00:43:33.24422+...	postgres

	_values nb	new_values jsonb
1	tatus": "active", "end_date": null, "tariff_id": 3, "start_date": "2026-10-16", "contract_id": 1, "subscriber_i...	{"status": "suspended", "end_date": null, "tariff_id": 3, "start_date": "2026-10-16", "contract_id": 1, "subscriber_i

5. Автоматический пересчёт итоговой стоимости договора

Добавить в таблицу contracts денормализованное поле total_monthly_cost и создать триггер, который будет автоматически пересчитывать его при любом изменении подключённых услуг (добавление, удаление, изменение). Это избавляет приложение от необходимости каждый раз выполнять сложные JOIN-запросы с агрегацией для получения текущей цены договора.

Добавление денормализованного поля и инициализация данных

```
ALTER TABLE contracts
ADD COLUMN IF NOT EXISTS total_monthly_cost NUMERIC(10, 2)
DEFAULT 0;

UPDATE contracts c
SET total_monthly_cost = t.monthly_price + COALESCE(
    (SELECT SUM(s.monthly_price)
     FROM contract_services cs
     JOIN services s ON cs.service_id = s.service_id
     WHERE cs.contract_id = c.contract_id),
    0
)
FROM tariffs t
WHERE c.tariff_id = t.tariff_id;

SELECT contract_id, tariff_id, total_monthly_cost
FROM contracts
WHERE contract_id IN (1, 3, 4);
```

Создание триггерной функции

```
CREATE OR REPLACE FUNCTION fn_recalc_contract_cost()
RETURNS TRIGGER AS $$
```

```
DECLARE
    v_contract_id INTEGER;
    v_tariff_price NUMERIC(10, 2);
    v_services_sum NUMERIC(10, 2);
BEGIN
    IF TG_OP = 'DELETE' THEN
        v_contract_id := OLD.contract_id;
    ELSE
        v_contract_id := NEW.contract_id;
    END IF;

    SELECT t.monthly_price INTO v_tariff_price
    FROM contracts c
    JOIN tariffs t ON c.tariff_id = t.tariff_id
    WHERE c.contract_id = v_contract_id;

    SELECT COALESCE(SUM(s.monthly_price), 0) INTO v_services_sum
    FROM contract_services cs
    JOIN services s ON cs.service_id = s.service_id
    WHERE cs.contract_id = v_contract_id;

    UPDATE contracts
    SET total_monthly_cost = v_tariff_price + v_services_sum
    WHERE contract_id = v_contract_id;

    IF TG_OP = 'DELETE' THEN
        RETURN OLD;
    ELSE
        RETURN NEW;
    END IF;
END;
$$ LANGUAGE plpgsql;
```

Создание триггера

```
CREATE TRIGGER trg_recalc_cost
AFTER INSERT OR UPDATE OR DELETE ON contract_services
FOR EACH ROW
EXECUTE FUNCTION fn_recalc_contract_cost();
```

Демонстрация работы

Добавление услуги

```
INSERT INTO contract_services (contract_id, service_id, connected_at)
VALUES (3, 2, CURRENT_TIMESTAMP);

SELECT contract_id, tariff_id, total_monthly_cost
FROM contracts
WHERE contract_id = 3;
```

	contract_id [PK] integer	tariff_id integer	total_monthly_cost numeric (10,2)
1	3	2	750.00

Удаление услуги

```
DELETE FROM contract_services
WHERE contract_id = 3 AND service_id = 2;

SELECT contract_id, tariff_id, total_monthly_cost
FROM contracts
WHERE contract_id = 3;
```

	contract_id [PK] integer	tariff_id integer	total_monthly_cost numeric (10,2)
1	3	2	600.00

6. Обоснование

В базе данных стоит размещать ту логику, которая критически важна для целостности данных и должна работать всегда. Например, триггер для пересчёта итоговой стоимости договора гарантирует актуальность денормализованного поля, а триггер аудита записывает все изменения в

журнал — обойти их невозможно. Также в БД имеет смысл держать атомарные операции: моя функция подключения услуги проверяет статус договора, отсутствие дубликатов и добавляет запись в одной транзакции, что исключает риск некорректного промежуточного состояния.

В приложении лучше размещать логику, которая часто меняется по требованиям бизнеса, например правила начисления скидок — её проще обновлять в коде, чем переписывать функции в базе данных. Также в приложении должна жить логика, связанная с внешними сервисами: отправка SMS или запросы к платёжным системам, чтобы база данных не зависела от внешних сетей. Валидацию пользовательского ввода, например проверку формата телефона, тоже лучше делать на стороне приложения для мгновенной обратной связи. В итоге триггеры и функция подключения услуги живут в БД, так как гарантируют корректность данных, а всё, что зависит от внешних факторов и часто меняется, остаётся в приложении.

Вывод

В ходе выполнения лабораторной работы №5 были освоены механизмы обеспечения целостности данных и реализации серверной бизнес-логики в СУБД PostgreSQL на примере предметной области «Интернет-провайдер».